

Journal de Traçabilité et Guide de Dépannage : LISF / Noah-MP (Bassin Allal El Fassi)

Auteur: M. EL Aabaribaoune (@um6p)

Ce document a été conçu pour conserver la **traçabilité** de l'ensemble des défis techniques rencontrés lors de l'implémentation du framework LISF (LIS, LDT, LVT) sur le cluster HPC Toubkal. Il est rédigé de manière pédagogique afin qu'un **utilisateur débutant** puisse comprendre les efforts fournis, reproduire les solutions, et s'approprier l'architecture du projet sans rester bloqué sur des problèmes techniques complexes.

1. Environnement et Compilation du Code Source LISF

Problème 1.1 : Conflits avec les Compilateurs Modernes (GCC 13/14)

- **Contexte** : Lors de la compilation du code source avec la commande `make`, nous avons utilisé le compilateur GNU Fortran (`gfortran`).
- **Le Problème** : Des erreurs fatales sont apparues, indiquant une "incompatibilité d'interface" pour des fonctions comme `get_command_argument`, `command_argument_count` et `system`.
- **L'Explication** : Le code source de LIS est ancien ("legacy"). À l'époque de sa création, certaines fonctions permettant de lire les arguments de la ligne de commande ou d'exécuter des commandes système n'étaient pas standards. Les développeurs de LIS avaient donc dû créer leurs propres "interfaces explicites" pour les définir manuellement. Aujourd'hui, avec les compilateurs récents (GCC 13 et plus), ces fonctions sont devenues des standards intrinsèques (incluses par défaut). Le compilateur panique car il voit la définition par défaut entrer en conflit avec la redéfinition manuelle du code LIS.
- **La Solution (Pas-à-pas)** : Pour régler ce problème, il faut dire au code LIS de ne pas utiliser ses propres définitions et de laisser le compilateur gérer.
- Ouvrir le fichier `lisf/lis/core/LIS_misc.h`.

- Mettre en commentaire (avec le symbole `!`) les blocs d'interface `interface get_command_argument` et `interface command_argument_count`.
- Ouvrir le fichier `lisf/ldt/WSF_OPL/LDT_wsf_oplMod.F90`.
- Commenter l'instruction `external :: system`.

Problème 1.2 : Édition des Liens (Linker) pour NetCDF et ESMF

- **Contexte** : Une fois le code compilé, le système tente de relier (lier) tous les morceaux pour créer les exécutables finaux (`LIS` , `LDT` , `LVT`).
- **Le Problème** : Des erreurs du type `"undefined reference to nf90_def_var"` apparaissent, ce qui signifie que le compilateur ne trouve pas les bibliothèques NetCDF-Fortran.
- **L'Explication** : Sur un cluster HPC comme Toubkal (utilisant EasyBuild/Lmod), les chemins vers les bibliothèques (comme ESMF ou NetCDF) sont chargés dans des variables d'environnement (`$EBROOTNETCDFMINFORTRAN`), mais le script de compilation automatique (`./configure`) ne les détecte pas correctement et oublie de les inclure dans l'édition des liens.
- **La Solution** : Après avoir exécuté `./configure`, il faut modifier manuellement les fichiers générés (`configure.lis`, `configure.ldt` et `configure.lvt`). Dans ces fichiers, on doit ajouter les drapeaux `-lnetcdff` (qui indique de lier NetCDF Fortran) et s'assurer que les variables `FFLAGS` et `LDFLAGS` pointent bien vers les chemins fournis par les modules HPC.

2. Automatisation et Téléchargement des Données

Problème 2.1 : Gestion des Identifiants NASA Earthdata

- **Contexte** : Le projet nécessite des forçages météorologiques (MERRA-2) et des données satellitaires (SMAP) hébergées par la NASA.
- **Le Problème** : Essayer de télécharger ces données directement dans le terminal avec la commande `wget <URL>` échouait avec une erreur `401 Unauthorized`.
- **L'Explication** : La NASA exige une authentification pour télécharger ses données. Lorsqu'on accède à un lien, la NASA nous redirige vers "Earthdata Login". Des outils simples comme `wget` gèrent mal ces redirections complexes avec cookies.
- **La Solution** :

- Création d'un fichier caché nommé `~/.netrc` dans le dossier utilisateur, contenant les identifiants en clair pour permettre une connexion automatique.
- Développement de **scripts Python dédiés** (`download_merra2.py` , `download_smap.py`). Ces scripts utilisent la librairie `requests` qui gère parfaitement les sessions, les cookies et les redirections. Ils téléchargent les données par "morceaux" (chunking) pour ne pas saturer la mémoire (RAM) avec des fichiers de plusieurs centaines de mégaoctets.

Problème 2.2 : Interruption des Téléchargements en Arrière-plan

- **Contexte** : Les téléchargements prenaient beaucoup de temps (plusieurs heures). Nous les avons donc lancés en arrière-plan en ajoutant un `&` à la fin de la commande (`python3 script.py &`).
 - **Le Problème** : En revenant plus tard, l'exécution suivante (LDT) plantait car de nombreux fichiers (comme `gfrac_jan.asc`) étaient totalement absents, et d'autres étaient corrompus ou téléchargés à moitié (ex: `topsoil30snew` bloqué à 63%).
 - **L'Explication** : Lorsque l'on lance un programme en arrière-plan sur un nœud de connexion d'un cluster, ce programme reste rattaché à la session SSH actuelle (la fenêtre du terminal). Si on ferme le terminal ou que la connexion internet saute, le serveur HPC envoie un signal "SIGHUP" (Hangup) qui "tue" instantanément tous les processus liés à cette session.
 - **La Solution** :
 - Nous avons codé une intelligence dans les scripts Python : avant de télécharger, le script vérifie si le fichier existe déjà et si sa taille correspond à la taille attendue. Si c'est le cas, il le passe. Si le fichier est coupé à la moitié, il reprend le téléchargement.
 - Nous avons relancé les commandes en utilisant `nohup` (No Hang Up) : `nohup python3 download_merra2.py >> log.txt 2>&1 &` . Cette commande immunise le script contre les déconnexions, garantissant qu'il s'exécute jusqu'au bout.
-

3. Configuration et Exécution de LDT (Génération des Paramètres)

Problème 3.1 : Bibliothèques Dynamiques (Shared Libraries) Introuvables

- **Contexte** : Exécution du programme `LDT` sur un nœud de calcul via SLURM (la commande `sbatch`).
- **Le Problème** : Le job s'arrêtait immédiatement. Le fichier d'erreur (`slurm_ldt_xxx.err`) indiquait : `error while loading shared libraries: libesmf.so: cannot open shared object file: No such file or directory` .
- **L'Explication** : Sur un cluster, les logiciels ne sont pas installés par défaut. Il faut les "charger" via la commande `module load` . Même si nous avons chargé ces modules sur le nœud de connexion pour compiler LDT, les nœuds de calcul (ceux qui exécutent vraiment la tâche) démarrent dans un environnement vide.
- **La Solution** : Il faut systématiquement ajouter le chargement des environnements au début de tout fichier script SLURM (ex: `job_1_ldt.sh` , `job_2_lis_opl.sh`).
Exemple ajouté : `module purge module load foss/2024a module load netCDF-Fortran/4.6.1`

Problème 3.2 : Erreur de Valeur de Remplissage (Fill Value) pour Landcover

- **Le Problème** : LDT s'est arrêté avec l'erreur : `[ERR] Landcover fill value: option not specified in the config file` .
- **L'Explication** : LDT crée une carte sur notre bassin Allal El Fassi. Parfois, il y a des pixels "vides" (donnée manquante) dans les données globales. Dans le fichier `ldt.config` , nous avons choisi l'option `neighbor` : si un pixel est vide, LDT regarde ses voisins pour lui donner la même valeur. Mais LDT exige une sécurité absolue : "Et si aucun des voisins n'a de valeur, que fais-je ?".
- **La Solution** : Ajout d'une ligne de secours dans `ldt.config` : `Landcover fill value: 10` . Le chiffre 10 correspond à la classe "Prairies/Grasslands" dans la classification internationale IGBP, ce qui est une approximation sûre et réaliste pour notre région si un pixel pose problème.

Problème 3.3 : Le Bug Obscur de la Topographie "GTPO30-Native"

- **Contexte** : LDT doit lire l'élévation du terrain (topographie). Nous avons téléchargé la tuile GTPO30 `W020N40.DEM`, qui couvre parfaitement l'Europe et le Maroc.
- **Le Problème** : LDT a planté avec l'erreur : `[ERR] GTPO30-Native elevation map file, ./input/topo_parms/GTPO30/gt30w140n40.dem, not found.` (Notez qu'il demande `w140`, une coordonnée située près de l'Amérique du Nord !).
- **L'Explication (Très importante)** : C'est un défaut de programmation rigide à l'intérieur du code source de LDT (`read_GTPO30Native_elev.F90`). Au lieu de regarder où se situe notre bassin et de n'ouvrir que la tuile correspondante, LDT est programmé pour faire une boucle sur les **33 tuiles mondiales** de GTPO30. Il vérifie que les 33 fichiers couvrant le globe terrestre entier existent bien dans le dossier. S'il en manque un seul, LDT panique et s'arrête, même si cette tuile ne concerne pas notre domaine de simulation !
- **La Solution (L'astuce "Python Symlinks")** : Télécharger la terre entière représentait des dizaines de gigaoctets totalement inutiles. Pour résoudre ce problème de manière élégante, nous avons créé un petit programme (`fix_gtopo.py`). Ce programme a généré des **Liens Symboliques** (l'équivalent des "raccourcis" sous Windows). Il a créé virtuellement les noms des 33 tuiles mondiales (ex: `gt30w140n40.dem`), mais toutes ces fausses tuiles pointent vers le vrai et unique fichier `W020N40.DEM` que nous avons téléchargé. Conséquence : LDT "voit" les 33 fichiers, est rassuré, et commence l'extraction. Au moment d'extraire la zone du Maroc, il tape dans la bonne donnée, et le reste des fausses tuiles est simplement ignoré !

Problème 3.4 : Paramètres Pente (Slope) et Orientation (Aspect) Incompatibles

- **Le Problème** : `ldt.config` exigeait des fichiers de pente et d'orientation via `GTPO30_Native`.
- **L'Explication** : Bien que GTPO30 fournisse l'élévation, les développeurs de LDT n'ont jamais codé la fonctionnalité permettant de calculer ou de lire la pente à partir du format `GTPO30_Native`. Le laisser dans la configuration crée un comportement erratique. De plus, le modèle de surface de notre choix (Noah-MP) utilise déjà sa propre carte globale de pente (`islope`).

- **La Solution** : Suppression complète des blocs `Slope data source` et `Aspect data source` dans le fichier `ldt.config` . Cela force LDT et Noah-MP à utiliser leurs comportements par défaut de manière propre, sans chercher des informations qui n'existent pas.
-

4. Exécution des Simulations (LIS Open-Loop et Data Assimilation)

Problème 4.1 : Erreur MPI_ABORT au lancement de LIS

- **Contexte** : Soumission des scripts d'exécution finaux (`job_2_lis_opl.sh` et `job_3_lis_da.sh`) sur le nœud de calcul.
- **Le Problème** : Le fichier de log (`lis_opl_run.log`) affichait immédiatement l'erreur fatale : `MPI_ABORT was invoked on rank 0 in communicator` . Le fichier de diagnostic `lislog.0000` n'était même pas créé.
- **L'Explication** : Dans les scripts originaux, la commande de lancement était écrite `.../LIS lis.config.opl` . Contrairement à LDT (qui accepte le fichier de config directement comme argument), l'exécutable LIS exige **strictement** le drapeau `-f` (ou `--file`) pour charger un fichier de configuration avec un nom non-standard. Sans ce `-f` , LIS a ignoré notre fichier et a cherché un fichier par défaut `lis.config` introuvable, provoquant un arrêt d'urgence du processus MPI.
- **La Solution** : Correction de la syntaxe de la ligne de commande dans les fichiers `.sh` pour y inclure le `-`

```
f : ./lisf/lis/LIS -f lis.config.opl > lis_opl_run.log 2>&1 ./lisf/lis/LIS  
-f lis.config.da > lis_da_run.log 2>&1
```

Problème 4.2 : LIS plante avec "Forcing perturbation algorithm: not defined"

- **Contexte** : Exécution de la simulation Open-Loop.
- **Le Problème** : Le job s'est arrêté avec l'erreur `[ERR] Forcing perturbation algorithm: not defined Stopping.` dans le fichier de diagnostic `lislog.0000` .
- **L'Explication** : Bien que la simulation Open-Loop n'utilise pas d'Assimilation de Données (DA) et ne perturbe pas les forçages, le moteur interne de LIS exige que

les algorithmes de perturbation soient explicitement désactivés dans le fichier de configuration.

- **La Solution** : Ajout des lignes suivantes dans `lis.config.opl` : `Forcing perturbation algorithm: "none"` `State perturbation algorithm: "none"` `Observation perturbation algorithm: "none"`

Problème 4.3 : LIS plante avec "Error in nf90_inq_dimid in read_slope"

- **Contexte** : Exécution des simulations LIS (Open-Loop et DA).
 - **Le Problème** : Le job s'est arrêté avec l'erreur `[ERR] Error in nf90_inq_dimid in read_slope Stopping.` dans le fichier `lislog.0000`.
 - **L'Explication** : Dans les fichiers de configuration de LIS (`lis.config.opl` et `lis.config.da`), les paramètres `Slope data source` et `Aspect data source` étaient réglés sur `LDT`. Cela demandait à LIS de lire les cartes de pente et d'orientation générées par LDT dans le fichier `lis_input.d01.nc`. Or, souvenez-vous du Problème 3.4 : nous avons supprimé la pente et l'orientation dans LDT à cause de l'incompatibilité de GTOPO30. LIS cherchait donc des variables qui n'existaient pas.
 - **La Solution** : Dans les deux fichiers de configuration de LIS (`lis.config.opl` et `lis.config.da`), nous avons remplacé : `Slope data source: none` `Aspect data source: none` (Note : Le modèle Noah-MP a de toute façon déjà accès à la classe de pente via la variable `islope` incluse dans les paramètres).
-

5. Ultimes Corrections de Configuration LIS

Problème 5.1 : "Perturbations start mode: not specified"

- **Contexte** : Après avoir fixé les algorithmes de perturbation à "none" pour Open-Loop, le job a de nouveau planté avec l'erreur `[ERR] Perturbations start mode: not specified Stopping.`
- **L'Explication** : Le parseur de configuration de LIS est extrêmement pointilleux. Même si nous lui disons de ne pas perturber les forçages (car c'est un Open-Loop

sans assimilation), il exige de trouver les définitions complètes du bloc "Perturbation options".

- **La Solution** : Ajout du bloc de texte fictif dans `lis.config.opl` : `Perturbations start mode: "coldstart" Perturbations restart output interval: "1mo" Perturbations restart filename: "none"`

Problème 5.2 : "Model output attributes file does not exist"

- **Contexte** : Exécution de la simulation Data Assimilation.
- **Le Problème** : Arrêt immédiat avec l'erreur `[ERR] Model output attributes file does not exist...`.
- **L'Explication** : À la ligne `Model output attributes file:` de nos fichiers `lis.config`, nous demandons de lire le fichier `./MODEL_OUTPUT_LIST.TBL`. Ce fichier contrôle exactement quelles variables physiques (Humidité du sol, Évapotranspiration, Température) doivent être écrites dans les NetCDF finaux. Mais nous avons oublié de le copier depuis les fichiers sources de LIS vers notre dossier de travail !
- **La Solution** : Copie du fichier depuis les modèles de test de la NASA vers la racine du projet : `cp lisf/lis/testcases/dataassim/nasa_smap_enkf_noah36/MODEL_OUTPUT_LIST.TBL ./`

6. Corrections des Tables de Paramètres Noah-MP (VEGPARM.TBL et MPTABLE.TBL)

Problème 6.1 : Erreur Fortran "Bad integer for item 1 in list input" dans VEGPARM.TBL

- **Contexte** : Après avoir résolu les problèmes de configuration LIS, le modèle s'est arrêté avec une erreur Fortran lors de la lecture du fichier `input/noah_2dparms/VEGPARM.TBL`.
- **Le Problème** : `Fortran runtime error: Bad integer for item 1 in list input` à la ligne correspondant à la lecture du fichier de paramètres de végétation.
- **L'Explication** : Le fichier `VEGPARM.TBL` téléchargé depuis le portail LIS contenait des lignes supplémentaires non reconnues par Noah-MP 3.6. Après la table standard USGS, il y avait un bloc de valeurs spéciales (`CROP: 3` ,

`LOW_DENSITY_RESIDENTIAL: 31` , `HIGH_DENSITY_RESIDENTIAL: 32` , etc.) qui correspondent à une classification étendue utilisée dans des versions plus récentes de Noah-MP (4.0+). Le parseur Fortran de Noah-MP 3.6, qui s'attend à lire une liste de chiffres entiers simples après les métadonnées, rencontrait ces étiquettes textuelles et échouait.

- **La Solution :**

- Identifier les lignes parasites dans VEGPARAM.TBL (les lignes après `NATURAL` et avant `Vegetation Parameters MODIFIED_IGBP_MODIS_NOAH`).
- Les supprimer avec la commande `sed` : `sed -i '/^CROP/,+7d' input/noah_2dparms/VEGPARM.TBL`
- Vérifier visuellement la structure résultante avec `head -n 50 input/noah_2dparms/VEGPARM.TBL` pour confirmer que les sections USGS et MODIS se suivent proprement.

Problème 6.2 : Erreur Fortran "End of file" puis "Cannot match namelist object" dans MPTABLE.TBL

- **Contexte** : Après avoir corrigé VEGPARAM.TBL, le modèle progressait puis se bloquait sur `MPTABLE.TBL` avec l'erreur `Fortran runtime error: End of file` (ligne 463 de `module_sf_noahmplsm_36.F90`).
- **Le Problème** : Noah-MP 3.6 lit `MPTABLE.TBL` en utilisant des **namelists Fortran** (blocs `&nom ... /`). Les noms de ces blocs que le code cherche sont précis : `&noah_mp_usgs_veg_categories` , `&noah_mp_usgs_parameters` , `&noah_mp_modis_veg_categories` , etc. Le fichier téléchargé depuis le répertoire de configuration 557WW utilisait la convention `&noahmp...` (sans le `_mp_`), que Noah-MP 3.6 ne sait pas trouver, d'où un "End of File" car il cherche sans jamais trouver.
- **Deuxième erreur** : Après avoir tenté une correction partielle, une nouvelle erreur `Cannot match namelist object name isice` est apparue. Cela signifie que le fichier MPTABLE contenait des clés récentes (`ISICE` , `ISCROP` , `MFSNO` , `NR00T` , `RGL` , etc.) absentes de la définition Fortran de Noah-MP 3.6. De plus, les tableaux 2D (ex: `RHOL`) étaient décomposés en `RHOL_VIS=` et `RHOL_NIR=` — une syntaxe que Fortran ne peut pas lire comme un seul tableau 2D. Les tableaux mensuels SAI et LAI étaient aussi découpés en 12 lignes séparées (`SAI_JAN=` , `SAI_FEB=` , ...) au lieu d'être concaténés sous une seule clé `SAIM=` .

- **L'Explication Technique (Versioning)** : Le fichier `MPTABLE.TBL` provenant des configurations 557WW (Noah-MP 4.0+) est **incompatible** avec le code Noah-MP 3.6 compilé dans LIS. Les deux versions n'utilisent ni les mêmes noms de namelist, ni les mêmes variables.
- **La Solution** :
 - Re-télécharger le MPTABLE original (version WRF-3, compatible avec Noah-MP 3.6) depuis GitHub NCAR : `wget -q0 input/noah_2dparms/MPTABLE.TBL https://raw.githubusercontent.com/NCAR/WRFV3/master/run/MPTABLE.TBL`
 - Appliquer un script Python (`fix_mptable.py`) qui effectue les transformations nécessaires :
 - Renommer les namelists : `&noahmp_` → `&noah_mp_`
 - Supprimer les clés inconnues de Noah-MP 3.6 : `ISICE` , `ISCROP` , `NATURAL` , `LOW_DENSITY_RESIDENTIAL` , `HIGH_DENSITY_RESIDENTIAL` , `HIGH_INTENSITY_INDUSTRIAL` , `MFSNO` , `NROOT` , `RGL` , `RS` , `HS` , `TOPT` , `RSMAX`
 - Fusionner les tableaux 2D : `RHOL_VIS=...` + `RHOL_NIR=...` → `RHOL=...<toutes valeurs>` (idem pour `RHOS` , `TAUL` , `TAUS`)
 - Fusionner les tableaux mensuels : `SAI_JAN=` , `SAI_FEB=` , ... `SAI_DEC=` → `SAIM=...<toutes valeurs>` (idem pour `LAI` → `LAIM`)
 - Fusionner les tableaux EPS : `EPS1=` , ..., `EPS5=` → `EPS=...<toutes valeurs>`
 - Commande : `python3 fix_mptable.py`

7. Erreur de Configuration : "Radiative transfer model: not defined"

Problème 7.1 : LIS s'arrête après l'initialisation de Noah-MP avec "Radiative transfer model: not defined"

- **Contexte** : Après avoir passé avec succès la phase de lecture des tables de paramètres (VEGPARM, SOILPARM, MPTABLE), LIS s'arrêtait proprement, juste après le coldstart de Noah-MP, avec : `[ERR] Radiative transfer model: not defined Stopping.`
- **Le Problème** : La ligne `Radiative transfer model:` était absente des deux fichiers de configuration `lis.config.opl` et `lis.config.da` .

- **L'Explication** : LIS intègre une architecture permettant de coupler le modèle de surface avec un "Modèle de Transfert Radiatif" (RTM), comme CRTM ou CMEM3. Ce champ est **obligatoire** même si on ne veut pas utiliser de RTM — LIS n'a pas de valeur par défaut pour ce paramètre. Si le champ est absent, le parseur interne de LIS lève immédiatement une erreur fatale. Pour la simulation Noah-MP 3.6 standard (sans RTM couplé), la valeur correcte est simplement `"none"`.
- **La Solution** : Ajouter la ligne suivante dans les deux fichiers de configuration, immédiatement après `Land surface model:` :
`Radiative transfer model: "none"` Fichiers modifiés :
 - `lis.config.opl` — ligne 9
 - `lis.config.da` — ligne 9

Note pour les utilisateurs futurs : Si vous souhaitez utiliser le module CMEM3 (microwave forward model) pour simuler les observations satellitaires, changez `"none"` en `"CMEM3"` et ajoutez le bloc de configuration CMEM3 correspondant.

8. Problèmes liés aux Données de Forçage MERRA-2

Problème 8.1 : Paramètre manquant `"Number of application models"` dans `lis.config`

- **Contexte** : Après avoir corrigé le modèle de transfert radiatif, LIS s'est arrêté à nouveau avec : `[ERR] Number of application models: option not specified in the config file Stopping.`
- **L'Explication** : LIS possède un mécanisme permettant de coupler des "Application Models" (modèles d'application spécialisés). Même sans en utiliser, ce champ est **obligatoire** et doit être explicitement défini à `0`.
- **La Solution** : Ajouter la ligne dans les deux fichiers de configuration : `Number of application models: 0`

Problème 8.2 : Structure de Répertoires MERRA-2 incompatible avec LIS

- **Contexte** : Après avoir passé avec succès toute la phase d'initialisation de Noah-MP, LIS a planté à son premier pas de temps avec : `[ERR] ./input/MET_FORCING/MERRA2//MERRA2_400/Y2020/M05/MERRA2_400.tavg1_2d_slv_Nx.20200531.nc4 does not exist`
- **Le Problème** : Le lecteur MERRA-2 de LIS (codé dans `get_merra2.F90`) attend les fichiers dans une arborescence précise : `<base>/MERRA2_400/Y<AAAA>/M<MM>/<fichier>.nc4` Mais notre script de téléchargement (`download_merra2.py`) avait organisé les fichiers par type de collection : `<base>/M2T1NXFLX/<fichier>.nc4` et `<base>/M2T1NXSLV/<fichier>.nc4`
- **La Solution** : Création d'un script `reorganize_merra2.sh` qui crée des **liens symboliques** depuis la structure attendue par LIS vers les fichiers réels téléchargés. Cette approche évite de dupliquer des dizaines de Go de données.
`bash bash reorganize_merra2.sh`

Problème 8.3 : LIS a besoin des données du jour AVANT la date de démarrage (May 31)

- **Le Problème** : La simulation commence le 1er juin 2020. LIS lit les fichiers de forçage par interpolation temporelle : il a besoin du **fichier de la fin du jour précédent** (31 mai 2020, 23h45) pour pouvoir calculer la valeur au temps $t=0$ (1er juin 2020, 00h00). Nos données ne commencent que le 1er juin.
- **La Solution** : Créer des liens symboliques pointant le fichier du 31 mai vers le fichier du 1er juin. LIS utilisera la même donnée de chaque côté du premier pas de temps, ce qui est une approximation raisonnable pour le démarrage : `bash ln -sf "$(realpath input/MET_FORCING/MERRA2/M2T1NXSLV/MERRA2_400.tavg1_2d_slv_Nx.20200601.nc4)" \ "input/MET_FORCING/MERRA2/MERRA2_400/Y2020/M05/MERRA2_400.tavg1_2d_slv_Nx.20200531.nc4" # (idem pour flx et rad)` Ce comportement est intégré dans le script `reorganize_merra2.sh`.

Problème 8.4 : Collection MERRA-2 manquante — `rad_Nx` (Radiation)

- **Contexte** : Après avoir corrigé la structure des répertoires, LIS progressait mais s'arrêtait immédiatement après avoir lu les fichiers `slv` et `flx` avec : `[ERR] ./`

```
input/MET_FORCING/MERRA2//MERRA2_400/Y2020/M05/MERRA2_400.tavg1_2d_rad_Nx.20200531.nc4 does not exist
```

- **Le Problème** : LIS attend **trois collections MERRA-2** pour chaque jour :
- **M2T1NXSLV** → Variables météo en surface (Température, Humidité, Vent, Pression)
- **M2T1NXFLX** → Flux turbulents de surface (Precipitation, etc.)
- **M2T1NXRAD** → **Bilan radiatif** (Rayonnement solaire incident, longwave, etc.) Notre script de téléchargement initial (`download_merra2.py`) n'avait téléchargé que **SLV** et **FLX** . La collection **RAD** était manquante.
- **La Solution (2 étapes)** :
- **Mettre à jour `download_merra2.py`** : Ajout du téléchargement de la collection **M2T1NXRAD** depuis `https://data.gesdisc.earthdata.nasa.gov/data/MERRA2/M2T1NXRAD.5.12.4`
- **Relancer le téléchargement** en arrière-plan (les fichiers SLV et FLX déjà téléchargés seront sautés automatiquement) : `bash nohup python3 download_merra2.py >> download_merra2.log 2>&1 &`
- Une fois le téléchargement terminé, relancer `bash reorganize_merra2.sh` pour créer les liens symboliques des fichiers RAD dans la structure LIS, puis relancer les jobs.

⚠ **Note importante** : La collection RAD représente ~214 Mo par jour × 92 jours ≈ ~20 Go supplémentaires. Prévoir le temps et l'espace disque nécessaires avant de lancer.

Problème 8.5 : DA plante avec "Data assimilation observation domain file: not defined"

- **Contexte** : La simulation Open-Loop tournait correctement, mais la simulation Data Assimilation s'arrêtait immédiatement avec : `[ERR] Data assimilation observation domain file: not defined Stopping.`
- **L'Explication** : Pour l'assimilation de données, LIS doit savoir sur quelle grille les observations satellitaires (SMAP) seront projetées. Ce fichier de domaine d'observation définit la correspondance géographique entre les pixels SMAP et les cellules du modèle Noah-MP. Le paramètre était absent du fichier `lis.config.da` .

- **La Solution** : Ajouter la ligne dans `lis.config.da` , dans le bloc Data Assimilation : `Data assimilation observation domain file: ./lis_input.d01.nc` On réutilise le même fichier `lis_input.d01.nc` généré par LDT, car nos observations SMAP seront interpolées sur la même grille que le modèle.
-

9. Résolution de l'Assimilation de Données SMAP et Plantages Obscurs (Segmentation Fault)

Problème 9.1 : Plantage brutal (Segmentation Fault) lors de l'initialisation de l'assimilation SMAP

- **Contexte** : Lors du lancement de la simulation avec Assimilation de Données (DA), le programme s'arrêtait immédiatement en se crashant brutalement avec un `Segmentation fault (core dumped)` ou un crash MPI sans aucun message d'erreur visible dans le log diagnostic `lislog.0000` .
- **Le Problème** : Le fichier de configuration `lis.config.da` fourni contenait la ligne : `Data assimilation set: "NASA SMAP soil moisture"`
- **L'Explication Technique** : À l'intérieur du code Fortran de LIS (`LIS_DAobs_pluginMod.F90`), la routine d'enregistrement de l'assimilation SMAP est déclarée avec l'identifiant exact `"SMAP(NASA) soil moisture"` . Cependant, lors de la lecture du fichier de configuration, LIS utilise du code C pour rechercher le nom d'observation correspondant à notre choix via un appel `strcmp` . Comme `"NASA SMAP soil moisture"` (le choix du fichier de configuration) et `"SMAP(NASA) soil moisture"` (le nom attendu par le code source) ne correspondent pas exactement, LIS n'a pas pu lier la routine d'initialisation (setup) correspondante, ce qui a causé un déréférencement de pointeur nul et un crash immédiat (Segmentation Fault).
- **La Solution** : Remplacer la ligne correspondante dans `lis.config.da` par le nom d'identification officiel et rigide du code source LIS : `Data assimilation set: "SMAP(NASA) soil moisture"`

Problème 9.2 : Plantage "SMAP(NASA) soil moisture data directory: is missing"

- **Contexte** : Une fois le nom officiel de l'assimilation corrigé, LIS a recommencé à s'initialiser proprement mais s'est immédiatement arrêté avec le message : `[ERR] SMAP(NASA) soil moisture data directory: is missing Stopping.`
- **L'Explication** : Le parseur de LIS pour le module d'observations SMAP (`NASASMAPsm_Mod.F90`) s'attend à lire des clés de configuration préfixées par `"SMAP(NASA)"` plutôt que `"NASA SMAP"` . Par exemple, le fichier de configuration original définissait : `NASA SMAP soil moisture observation directory: ...` Mais le code source recherche strictement la clé : `SMAP(NASA) soil moisture data directory: ...`
- **La Solution** : Mettre à jour l'ensemble des clés obsolètes ou mal nommées dans `lis.config.da` :
- Remplacer `NASA SMAP soil moisture observation directory:` par `SMAP(NASA) soil moisture data directory:`
- Remplacer `NASA SMAP soil moisture data designation:` par `SMAP(NASA) soil moisture data designation:`
- Remplacer `NASA SMAP soil moisture Composite Release ID (e.g., R16):` par `SMAP(NASA) soil moisture Composite Release ID:`
- Remplacer `NASA SMAP use scaled standard deviation model:` par `SMAP(NASA) soil moisture use scaled standard deviation model:`

Problème 9.3 : Arrêt "SMAP(NASA) model CDF file: not defined Stopping"

- **Contexte** : Même après avoir corrigé les répertoires, LIS s'arrêtait avec des messages d'erreur indiquant que les fichiers CDF étaient manquants, bien que nous n'utilisions aucun algorithme de mise à l'échelle CDF (scaling strategy = "none").
- **L'Explication** : Dans le module SMAP de LIS (`NASASMAPsm_Mod.F90`), les instructions de vérification des paramètres CDF (recherche de la table de biais, du nombre de bins de la courbe CDF, et de l'option de lecture) sont écrites de manière inconditionnelle à l'extérieur des blocs conditionnels du choix de la stratégie de mise à l'échelle. Par conséquent, LIS vérifie et requiert la présence de

ces paramètres dans la configuration, même s'ils ne sont pas activement utilisés par la suite.

- **La Solution** : Ajouter des définitions par défaut ("dummy" ou factices) dans le fichier `lis.config.da` pour satisfaire le processus d'initialisation inconditionnel du code sans pour autant activer ces fonctions : `SMAP(NASA) model CDF file: none`
`SMAP(NASA) observation CDF file: none` `SMAP(NASA) soil moisture number of bins in the CDF: 0` `SMAP(NASA) CDF read option: 0`
-

10. Automatisation des Configurations et Corruption des Chemins

Problème 10.1 : Corruption des chemins cibles dans les fichiers de configuration générés

- **Contexte** : Lors de l'automatisation de la génération des configurations pour les tests de scalabilité, les chemins cibles dans `lis.config` et `ldt.config` ont été corrompus.
 - **Le Problème** : Des chemins comme `./data/lis_input.d01.nc` ont été transformés en `./dat./data/lis_input.d01.nc`, causant des erreurs d'initialisation fatales lors de l'exécution de LDT et LIS.
 - **L'Explication** : Le script d'automatisation `scripts/generate_configs.sh` utilisait des remplacements `sed` successifs et redondants. Le remplacement de `data/` par un autre chemin et de `./` par un autre a créé des collisions de motifs, appliquant les modifications deux fois.
 - **La Solution** : Corriger le script `scripts/generate_configs.sh` en veillant à ce que les motifs de remplacement soient uniques et n'entrent pas en collision (par exemple, en ciblant des expressions plus précises et en éliminant les lignes redondantes).
-

11. Soumission SLURM et Répertoire de Travail (\$SLURM_SUBMIT_DIR)

Problème 11.1 : Échec immédiat du Job de téléchargement (Exit Code 2)

- **Contexte** : Lors de la soumission du job SLURM de téléchargement (`scripts/jobs/job_download_data.sh`), le job s'est terminé instantanément avec un code de sortie 2, sans télécharger de données.
 - **Le Problème** : Le fichier de log d'erreur SLURM indiquait : `arch/arch_toubkal.env: No such file or directory` et `python3: can't open file 'scripts/download/...': [Errno 2] No such file or directory`.
 - **L'Explication** : L'utilisateur a soumis le job en étant positionné dans le dossier `scripts/jobs/` en exécutant `sbatch job_download_data.sh`. Par défaut, SLURM définit `$SLURM_SUBMIT_DIR` sur le répertoire depuis lequel la commande `sbatch` a été lancée. Le script exécutait `cd $SLURM_SUBMIT_DIR`, maintenant le répertoire de travail dans le sous-dossier `scripts/jobs`, où les répertoires `arch/` et `scripts/` n'existent pas.
 - **La Solution** : Remplacer le simple `cd $SLURM_SUBMIT_DIR` dans les scripts de jobs par un bloc de détection automatique robuste qui recherche le fichier d'environnement d'architecture `arch/arch_toubkal.env` dans les répertoires parents successifs et se déplace automatiquement à la racine du dépôt :

```
bash if [ -f "arch/arch_toubkal.env" ]; then cd . elif [ -f "../.. /arch/arch_toubkal.env" ]; then cd ../../ elif [ -f "../../arch/arch_toubkal.env" ]; then cd ../../ fi
```
-

12. Requêtes NASA CMR API et Recherche par Joker (Wildcard Search)

Problème 12.1 : Recherche de granules MODIS LAI retournant 0 résultat (Found 0 granules)

- **Contexte** : Lors de l'exécution de `download_modis_lai.py`, l'API NASA CMR retournait 0 granule trouvé pour la tuile `h17v05`, bien que le produit soit disponible pour la période cible.
 - **Le Problème** : Le dictionnaire des paramètres de requête HTTP comprenait `"readable_granule_name[]": "*h17v05*"` mais l'API de recherche traitait les astérisques de manière littérale, ne trouvant aucun nom de fichier contenant textuellement des astérisques.
 - **L'Explication** : Par défaut, l'API de recherche des granules de NASA CMR (Common Metadata Repository) traite les filtres textuels comme des correspondances littérales. Pour lui indiquer d'interpréter les jokers `*` et `?` dans les requêtes de motifs (pattern matching), il faut explicitement passer l'option booléenne `"options[readable_granule_name][pattern]": "true"`.
 - **La Solution** : Ajouter le paramètre d'option d'activation de motif dans la requête HTTP :

```
python params = { "short_name": "MOD15A2H", "version": "061",  
"temporal": f"{t_start},{t_end}", "readable_granule_name[]": f"*{TILE}*",  
"options[readable_granule_name][pattern]": "true", "page_size": 200,  
"page_num": page_num }
```
-

13. Assimilation de données conjointe (SMAP + MODIS LAI) et format des observations

Problème 13.1 : Routine d'initialisation manquante pour "MODIS LAI" (setup routine not defined)

- **Contexte** : Lors du lancement de la simulation de DA conjointe avec `job_3_lis_da_sebou.sh`, l'exécution plante immédiatement avec le message : `setup routine for DA obs MODIS LAI is not defined`.

- **Le Problème** : Le nom de jeu de données `"MODIS LAI"` n'est pas un nom de plugin reconnu/enregistré dans le code source de LIS (dans `src/lisf/lis/plugins/LIS_pluginIndices.F90`). Le plugin officiel pour le produit MODIS LAI s'appelle `"MCD15A2H LAI"`.
- **L'Explication** : LIS cherche la clé d'enregistrement `"MODIS LAI"` et, ne la trouvant pas, s'arrête avec une erreur de segmentation. De plus, le plugin `"MCD15A2H LAI"` s'attend à lire des fichiers NetCDF4 (`.nc4`) globaux (de dimensions `86400 x 43200` représentant une grille géographique de 500m de résolution de -180 à +180 de longitude et -90 à +90 de latitude) nommés `MCD15A2H.006_LAI_YYYYDOY.nc4`, alors que les fichiers téléchargés sont des dalles brutes au format HDF4 (`MOD15A2H.A*.hdf`) en projection sinusoïdale MODIS.
- **La Solution** :
 - Remplacer `"MODIS LAI"` par `"MCD15A2H LAI"` dans les fichiers de configuration `lis.config.da_sebou` et `lis.config.da_joint`, et configurer les paramètres de MCD15A2H spécifiques (version, QC flags, etc.).
 - Écrire un script de prétraitement Python (`scripts/fix/preprocess_modis_lai.py`) utilisant GDAL (`osgeo.gdal`) pour projeter la dalle `h17v05` en coordonnées géographiques (EPSG:4326) et l'insérer dans une grille globale de dimensions `86400 x 43200` compressée avec NetCDF4.

Problème 13.2 : Python `netCDF4` module non disponible dans l'environnement `arch_toubkal.env`

- **Contexte** : La première version du script de prétraitement utilisait `import netCDF4 as nc` → `ModuleNotFoundError: No module named 'netCDF4'`.
- **L'Explication** : L'environnement `arch_toubkal.env` charge des modules HPC compilés pour LIS/Fortran (foss/2024a, netCDF-Fortran, ESMF, etc.), mais les liaisons Python de netCDF4 ne sont pas incluses dans ces modules.
- **La Solution** : Réécrire le script pour n'utiliser que `osgeo.gdal` + `numpy` (disponibles via `module load GDAL/3.7.1-foss-2023a`) pour créer les fichiers NetCDF4, en exploitant le pilote GDAL `netCDF` avec l'option `FORMAT=NC4` et `COMPRESS=DEFLATE`.

Problème 13.3 : Noms de variables `Band1` / `Band2` au lieu de `Lai_500m` / `FparLai_QC`

- **Contexte** : Le pilote GDAL netCDF nomme automatiquement les bandes `Band1` , `Band2` , etc., alors que LIS (dans `read_MCD15A2H_LAI_data`) cherche explicitement les variables `Lai_500m` et `FparLai_QC` via `nf90_inq_varid` .
- **La Solution** : Utiliser `ncrename` de la suite NCO (disponible via `module load NCO/5.0.3-foss-2021b`) pour renommer les variables après la création GDAL. Le chemin complet `/srv/software/easybuild/software/NCO/5.0.3-foss-2021b/bin/ncrename` est codé en dur dans le script pour éviter les problèmes de variable `$PATH` dans les sous-processus Python : `bash ncrename -v Band1,Lai_500m -v Band2,FparLai_QC tmp_file.nc4 output.nc4`
- **Vérification** : Le fichier final est vérifié via `gdal.Open()` qui retourne les deux subdatasets `Lai_500m [43200x86400]` et `FparLai_QC [43200x86400]` .

Résultat final

- **Fichiers produits** : `data/observations/MODIS_LAI/processed/YYYY/MCD15A2H.006_LAI_YYYYD0Y.nc4`
- **Format** : NetCDF4, DEFLATE compressé (ZLEVEL=4), ~15 MB par fichier (1 dalle `h17v05` sur fond de `255`)
- **Job de preprocessing** : `scripts/jobs/job_preprocess_modis_lai.sh` (SLURM, nœud CPU unique, 12h)
- **Configurations mises à jour** : `configs/lis.config.da_sebou` , `configs/lis.config.da_joint`
- **Paramètres ajoutés** : `Data assimilation set: "SMAP(NASA) soil moisture"`
`"MCD15A2H LAI" MCD15A2H LAI data directory: ./data/observations/MODIS_LAI/processed`
`MCD15A2H LAI data version: 6 MCD15A2H LAI apply QC flags: 1`
`MCD15A2H LAI apply temporal smoother: 0` ``

15. Crash à `03:00` (Model Time) lors de l'Assimilation LAI (MCD15A2H)

Symptômes: - Les simulations `DA-LAI` et `DA-Joint` s'arrêtent net (crash) après 3 heures de simulation (généralement à `03:00` model time, le moment de la première perturbation d'état EnKF). - Le log `lislog.0000` s'arrête brusquement et le fichier standard output (SLURM) se remplit d'erreurs répétées : `Error in the return code,`

`Prgm Stopping...` suivies d'une fin d'exécution MPI. - Étrangement, un message d'avertissement apparaît plus tôt (à `00:00`) : `[WARN] Missing LAI file: VEGPARM.TBL ... ./data/land_params/noah_2dparms/` (nom de fichier contenant des caractères aléatoires / garbage memory).

Causes Techniques (2 Bugs distincts) : 1. Uninitialized Memory (Garbage String) pour le nom de fichier MODIS :

Dans les fichiers de configuration `lis.config.da_lai_sebou` et `lis.config.da_joint`, la version des données MCD15A2H était définie sur `6`. Cependant, le code source LIS (`read_MCD15A2Hlai.F90`) effectue une vérification stricte du format texte via l'instruction `if(version.eq."006")`. Si la condition n'est pas remplie, la variable `filename` n'est *jamais initialisée* et hérite du reste de mémoire (souvent le nom du précédent fichier de paramètre lu, ex: `VEGPARM.TBL`).

2. ESMF State Variables Mismatch pour la perturbation EnKF :

Dans nos fichiers de configuration des attributs de perturbation d'état (`data/pert_package/noahmp_lai_attribs.txt` et `noahmp_lai_pertattribs.txt`), nous avons défini le paramètre à perturber comme `Leaf Area Index`. Néanmoins, le modèle Noah-MP (dans `noahmp36_updatevegvars.F90`) n'enregistre pas la variable avec ce nom-là, mais l'enregistre avec le string exact `"LAI"`. À `03:00`, lorsque le module d'assimilation EnKF essaie de lire l'état pour le perturber, l'appel `ESMF_StateGet(LSM_State, "Leaf Area Index")` échoue car il ne trouve aucune variable avec ce nom. LIS détecte un code d'erreur de retour non nul et s'interrompt brusquement.

Résolution / Actions Mises en Place : 1. Fix de la Version MODIS :

Modification de `lis.config.da_lai_sebou` et `lis.config.da_joint` : `text MCD15A2H LAI data version: 006`

2. Fix de l'Attribut de Perturbation d'État :

Modification des fichiers `noahmp_lai_attribs.txt` et `noahmp_lai_pertattribs.txt` : Remplacement complet de `Leaf Area Index` par `LAI` : `text LAI 0.01 10.0` Et dans le fichier de perturbation : `text LAI 0 0.05 2.0 1 43200 0 0 0.0 0.0 0.0 0.0 1.0`

3. Suite à ces correctifs, les jobs ont été soumis à nouveau et ont complété leurs 3 jours de simulation avec succès.

Fin du journal. Ces documentations assurent la pérennité du projet et évitent de "réinventer la roue" ou de rester bloqué de longues heures sur des problèmes d'architecture lors des prochains travaux de recherche ou lors du passage de relais à un étudiant/chercheur.